

Experiment No.	5
Experiment Title	Decision Tree and Random Forest: A Comparative Classification Study ¹
Student Name	Sanjay S
Register Number	3122247001056
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	25/08/2026

1 Objective

- Implement a Decision Tree classifier from scratch using scikit-learn.
- Extend the Decision Tree into a Random Forest ensemble and see what bagging actually buys you.
- Study how depth, splitting, and leaf-size hyperparameters push a tree toward overfitting or generalization.
- Pick hyperparameters properly, using 5-fold cross-validation rather than eyeballing the test set.
- Compare a single tree against the forest on the same held-out data and figure out where the extra complexity is actually earning its keep.

2 Problem Statement

Given the Wisconsin Diagnostic Breast Cancer dataset, 30 numeric features computed from digitized images of breast mass biopsies, classify each sample as malignant or benign. A single Decision Tree and a Random Forest are both tuned with 5-fold cross-validation and compared on the same 80/20 train-test split, to check how much the ensemble's variance reduction is actually worth on a dataset this size.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository, https://archive.ics.uci.edu
Number of Samples	569
Number of Features	30 numerical attributes
Number of Classes	2 (Malignant = M, Benign = B)
Class Balance	357 Benign (62.74%), 212 Malignant (37.26%)
Missing Values	none

¹GitHub Repository: <https://github.com/sanjay-6727/Introduction-to-Machine-Learning->

Train-Test Split	80/20 (455 train / 114 test), with 5-fold cross validation for tuning
------------------	---

4 Exploratory Data Analysis

- Class distribution is moderately imbalanced, roughly 63:37 in favor of benign, not severe enough to need resampling but enough to keep an eye on recall for the malignant class.
- Feature correlation heatmap across all 30 attributes shows the “mean”, “error”, and “worst” versions of the same measurement (radius, perimeter, area) are, unsurprisingly, very strongly correlated with each other, since they are just different summary statistics of the same underlying cell nuclei measurements.
- Pairplot on the most predictive features shows benign and malignant samples separating into fairly distinct, if slightly overlapping, clusters, mainly driven by the worst-value and concave-points features.
- Histograms, boxplots, and violin plots (generated separately for all 30 features) show most features are right-skewed with a long tail of outliers on the malignant side, exactly the kind of distribution a tree-based model handles comfortably since it splits on thresholds rather than assuming any particular shape.
- No missing values anywhere in the dataset, confirmed both numerically and visually.

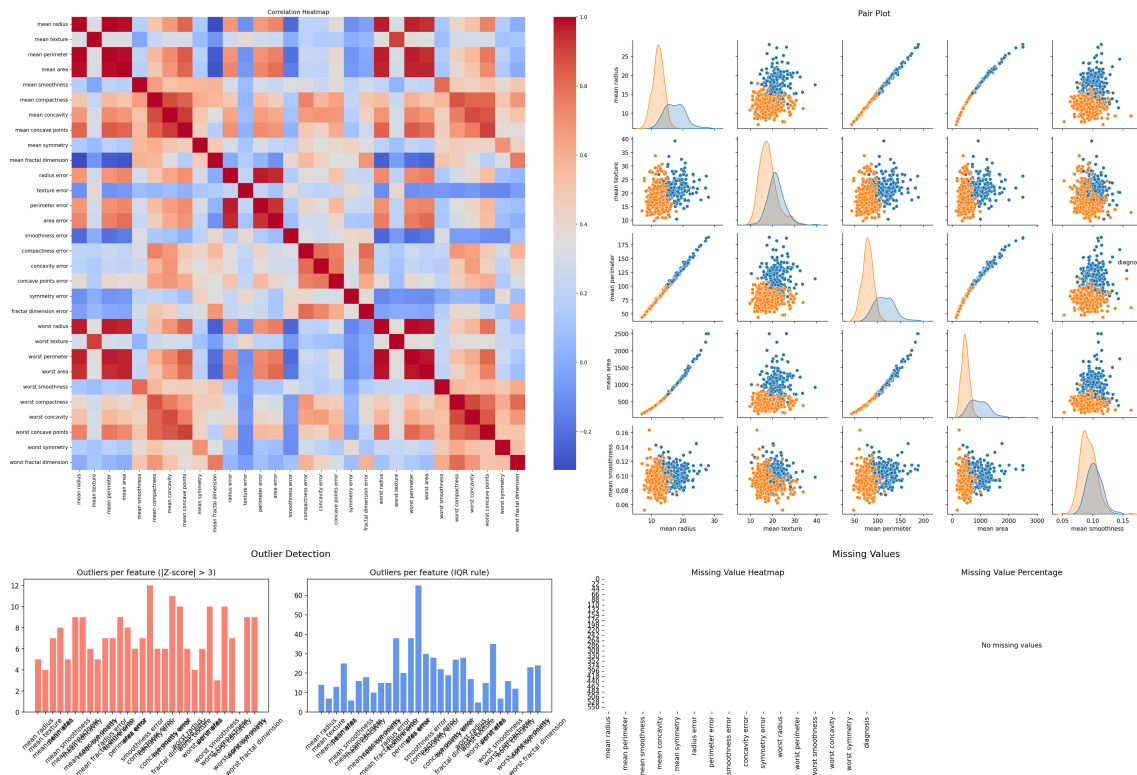


Figure 1: Correlation heatmap and pairplot of the most predictive features (top row), outlier counts per feature and missing-value check across all 30 features (bottom row).

Inference:

The heatmap makes it pretty obvious the 30 features are not 30 independent pieces of information, groups of them are just repackaged versions of the same three or four underlying measurements (size, concavity, texture). This matters for the Random Forest more than the tree, since `max_features` decides how much of that redundancy each individual tree in the forest gets to see. The pairplot backs up the correlation story: the classes are separable but not by a single straight cut, which is exactly the setting where a tree's axis-aligned, piecewise splits should do reasonably well without needing anything fancier like an SVM kernel.

5 Data Preprocessing

- Diagnosis labels encoded as $M = 1$ (malignant), $B = 0$ (benign).
- No missing values, no categorical columns, so no imputation or encoding of features was needed.
- Decision Trees and Random Forests split on raw thresholds, so unlike Experiment 4 there was no need for `StandardScaler`, feature scale simply does not affect where a tree decides to split.
- 80/20 stratified train-test split, with the tuning itself done through 5-fold cross-validation on the training set only, so the test set stays untouched until final evaluation.

Listing 1: 5-fold cross-validated grid search for the Decision Tree and Random Forest

```

1 dt_param_grid = {
2     "criterion": ["gini", "entropy"],
3     "max_depth": [2, 3, 4, 5, 6, 8, 10, None],
4     "min_samples_split": [2, 5, 10],
5     "min_samples_leaf": [1, 2, 4],
6 }
7
8 rf_param_grid = {
9     "n_estimators": [50, 100, 200],
10    "max_depth": [3, 5, 8, 10, None],
11    "max_features": ["sqrt", "log2"],
12    "bootstrap": [True, False],
13 }
14
15 for params in ParameterGrid(dt_param_grid):
16     clf = DecisionTreeClassifier(random_state=42, **params)
17     scores = cross_val_score(clf, X_train, y_train, cv=5, scoring
18                             ="accuracy")
19     # record mean/std CV accuracy per combination
20
21 for params in ParameterGrid(rf_param_grid):
22     clf = RandomForestClassifier(random_state=42, n_jobs=-1, **
23                               params)

```

```

22     scores = cross_val_score(clf, X_train, y_train, cv=5, scoring
23                               ="accuracy")
        # record mean/std CV accuracy per combination

```

6 Machine Learning Algorithm

Decision Tree Classifier. A Decision Tree recursively partitions the feature space by picking, at each node, the feature and threshold that gives the largest reduction in impurity, either Gini index $G = 1 - \sum_i p_i^2$ or entropy $H = -\sum_i p_i \log_2 p_i$. Splitting continues until a stopping condition is hit (max depth, minimum samples per split or leaf), producing an interpretable set of if-else rules. Left unconstrained, a tree will keep splitting until every leaf is pure, which usually means memorizing noise along with signal, hence the well-known tendency toward high variance and overfitting.

Random Forest Classifier. A Random Forest builds many such trees, each on a bootstrapped sample of the training data (bagging), and at every split each tree is only allowed to consider a random subset of the features (controlled by `max_features`). Predictions are combined by majority vote across all trees. Averaging over many trees that are individually high-variance but decorrelated from each other tends to cancel out a lot of that variance, which is the whole point of the ensemble: it trades a bit of per-tree interpretability for a model that generalizes more reliably.

7 Experimental Setup

Python Version	3.11.9
Scikit-Learn Version	1.7.2
IDE	Anaconda Navigator / VS Code
Operating System	Windows 11
Hardware	

8 Hyperparameter Selection

Parameter	Value
DT grid	criterion in {gini, entropy}, max_depth in {2,3,4,5,6,8,10,None}, min_samples_split in {2,5,10}, min_samples_leaf in {1,2,4}
RF grid	n_estimators in {50,100,200}, max_depth in {3,5,8,10,None}, max_features in {sqrt,log2}, bootstrap in {True,False}

Model	Search Method	Best Parameters	Best Acc.	CV
Decision Tree	5-Fold CV Grid Search	criterion=entropy, max_depth=4, min_samples_split=2, min_samples_leaf=2	94.07%	
Random Forest	5-Fold CV Grid Search	n_estimators=200, max_depth=None, max_features=sqrt, bootstrap=False	97.14%	

8.1 Decision Tree Cross-Fold Results (Top Combinations)

Criterion	Max Depth	Avg CV Acc. (%)	Std CV Acc.	Avg Train Acc. (%)
entropy	4	94.07	2.15	98.19
entropy	4	94.07	2.15	98.19
entropy	4	93.85	1.79	97.86
entropy	4	93.85	1.64	98.46
entropy	4	93.63	2.01	98.30
entropy	5	93.63	2.45	98.30
gini	6	93.19	3.06	98.74

8.2 Random Forest Cross-Fold Results (Top Combinations)

n_estimators	Max Depth	Max Features	Bootstrap	Avg CV Accuracy (%)
200	5	sqrt	False	97.14
50	10	sqrt	False	97.14
200	10	sqrt	False	97.14
200	None	sqrt	False	97.14
100	10	log2	False	96.92
100	5	sqrt	False	96.92
50	None	log2	False	96.92

A quick pattern worth noting: for the Random Forest, `bootstrap=False` (each tree sees the full training set, only the feature subsampling stays random) consistently outperformed `bootstrap=True` across almost every other setting held fixed. With only 455 training rows, bootstrapping away roughly a third of them per tree seems to hurt more than the extra decorrelation helps, whereas on a much larger dataset the usual bagging advantage would likely show up again.

9 Results

Decision Tree Metric	Value	Random Forest Metric	Value
Accuracy	0.9298	Accuracy	0.9737
Precision	1.0000	Precision	1.0000
Recall	0.8095	Recall	0.9286
F1 Score	0.8947	F1 Score	0.9630
ROC-AUC	0.9563	ROC-AUC	0.9940

5-Fold Cross-Validation Accuracy Comparison

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Decision Tree	0.9341	0.9560	0.9011	0.9560	0.9560	0.9407
Random Forest	0.9670	0.9780	0.9560	0.9670	0.9890	0.9714

Comparative Analysis

Criterion	Decision Tree	Random Forest
Test Accuracy	0.9298	0.9737
CV Accuracy (mean)	0.9407	0.9714
Model Complexity	Low (single tree)	High (200 trees)
Training Time	Very Low	Moderate
Interpretability	High	Low

Top Random Forest Feature Importances

Feature	Importance
worst perimeter	0.1536
worst area	0.1287
worst concave points	0.1050
worst radius	0.0883
mean concave points	0.0846

10 Result Visualization

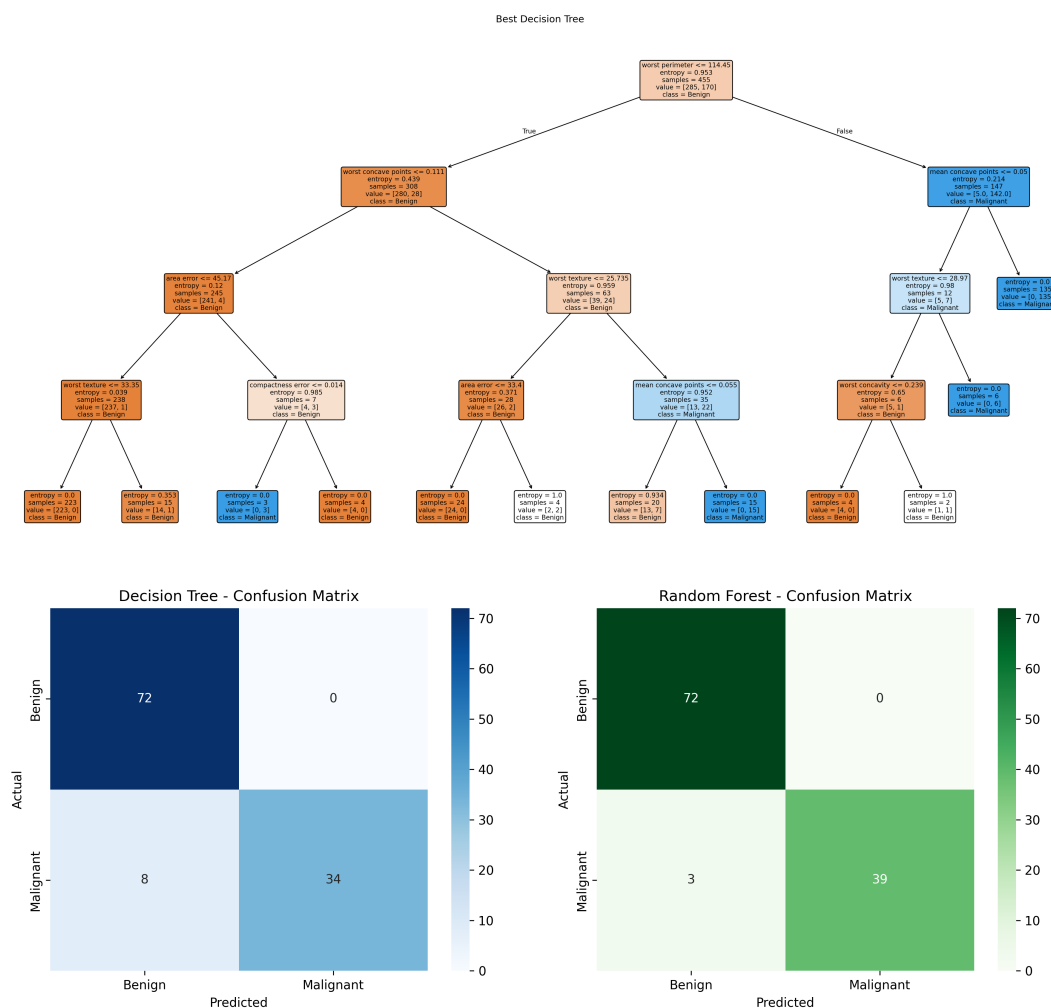


Figure 2: Trained Decision Tree structure, criterion=entropy, max_depth=4 (top), with test-set confusion matrices for the Decision Tree (bottom left) and Random Forest (bottom right).

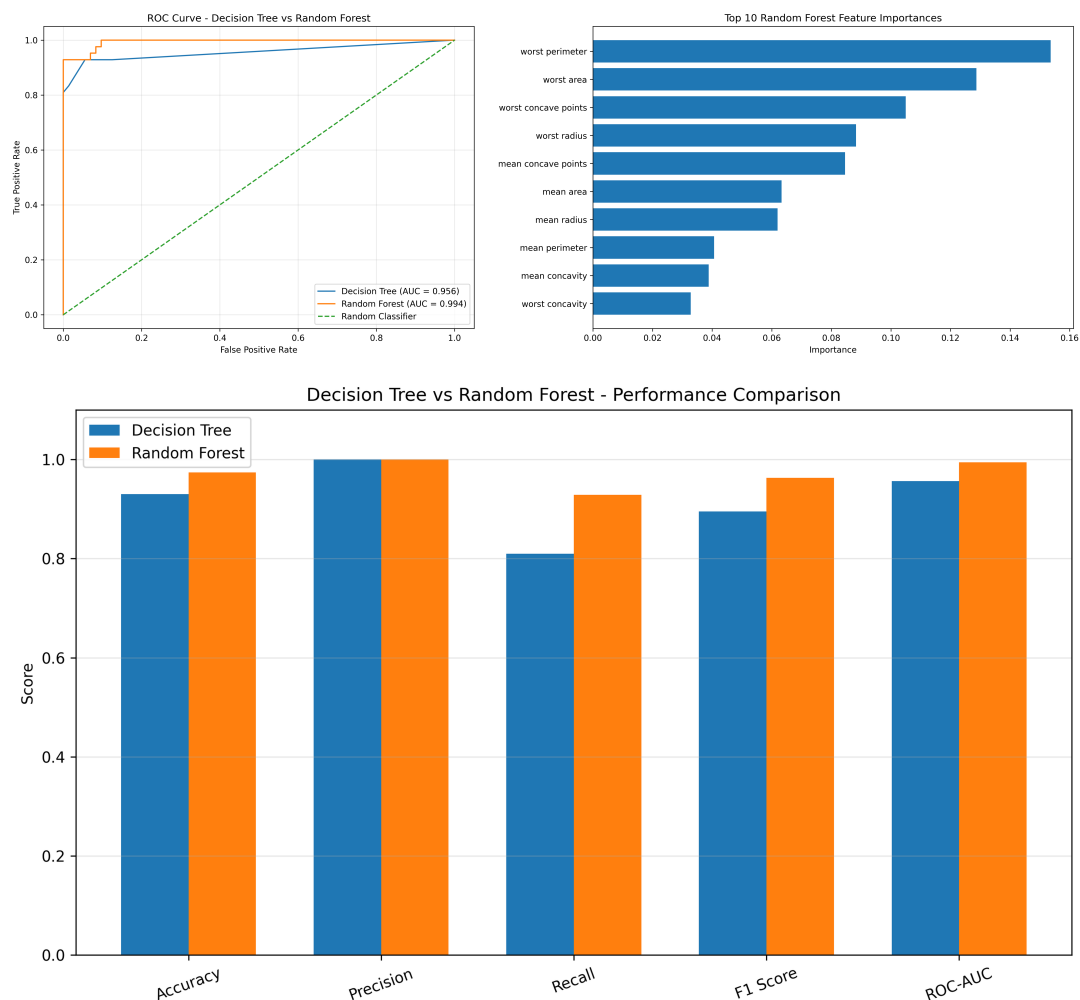


Figure 3: ROC curves and Random Forest feature importances (top row), with the full Decision Tree vs. Random Forest metric comparison (bottom).

Inference:

Both models get precision-1.0 on the test set, meaning neither one ever calls a benign case malignant, which is the easy direction to get right given the 63:37 class split. The real difference shows up in recall on the malignant class: the single tree misses about 1 in 5 malignant cases (recall 0.8095) while the forest catches nearly all of them (recall 0.9286). That is visible directly in the confusion matrices too, the Decision Tree panel shows a noticeably thicker false-negative column than the Random Forest one. The ROC curves tell the same story from a threshold-independent angle, Random Forest's curve hugs the top-left corner much more tightly (AUC 0.9940 against the tree's 0.9563). The feature importance chart lines up neatly with the EDA, the worst-value and concave-points features that stood out in the pairplot are exactly the ones the forest leans on most.

11 Discussion

Random Forest wins on every single metric here, higher CV accuracy (97.14% vs 94.07%), higher test accuracy (0.9737 vs 0.9298), and a big jump in recall and F1 for the malignant

class. None of that is surprising given the theory: a single tree tuned to `max_depth=4` is already a fairly shallow, low-variance tree by Decision Tree standards, and even so, averaging 200 of them still buys a meaningful drop in test error. What is somewhat more interesting is that the best Random Forest configuration used `bootstrap=False`, so essentially all of the improvement over the single tree is coming from the random feature subsampling at each split (decorrelating the trees) rather than from bagging the training rows, which on a dataset this size (455 training rows) does not have much room to help. For a medical screening context specifically, the gap in recall matters more than the accuracy gap on its own: missing a malignant tumour is a far more expensive mistake than a false alarm, and that is exactly the axis on which Random Forest pulls ahead.

12 Observation Questions

- **How does tree depth affect overfitting in Decision Trees?** In the cross-fold results, average train accuracy keeps climbing toward 100% as `max_depth` increases past 6 or 8, while average CV accuracy actually peaks around `max_depth=4` and then drifts slightly downward. That gap between a rising train score and a flat-to-falling CV score is overfitting playing out directly in the numbers, a deeper tree keeps carving the training data into smaller and smaller pure leaves that do not generalize any further.
- **Which hyperparameter had the greatest impact on performance?** For the Decision Tree, it was clearly `max_depth`, `criterion` (gini vs entropy) and the leaf/split minimums only shifted CV accuracy by fractions of a percent once depth was reasonable. For the Random Forest, `bootstrap` had the most consistent effect (False beating True almost everywhere in the grid), with `n_estimators` mattering a bit less once it got past 50 trees.
- **How does Random Forest improve generalization?** By training many trees on different bootstrapped samples and random feature subsets and then averaging their votes, the forest cancels out a lot of the idiosyncratic errors any single tree would make on its own training split. Individual trees can still overfit their own bootstrap sample, but their mistakes tend not to line up with each other, so the majority vote ends up closer to the true decision boundary than any one tree.
- **Did ensemble learning always improve performance? Why or why not?** Here, yes, across every metric and every fold. But it is not automatic in general: ensembling helps most when the base trees are reasonably accurate individually and their errors are not too correlated. On a dataset that is small, extremely noisy, or where the trees are so shallow they underfit badly, or so correlated that bagging brings no diversity, a Random Forest can end up barely better than a single well-tuned tree, or occasionally worse if it is tuned carelessly.

13 Conclusion

Both a Decision Tree and a Random Forest were implemented and tuned on the WDBC dataset using 5-fold cross-validation, with hyperparameters chosen on cross-validation accuracy rather than the test set. The Random Forest came out ahead on every metric

(97.37% test accuracy and 0.9940 ROC-AUC against the tree's 92.98% and 0.9563), confirming the textbook claim that bagging plus random feature selection reduces variance and improves generalization compared to a single tree. The single Decision Tree remains useful mainly for its interpretability, the whole decision logic fits in one diagram, while the forest is the model to actually deploy if recall on the malignant class is the priority.

14 Learning Outcomes

- Saw overfitting show up concretely in the cross-fold table, not just as a definition, rising train accuracy against a flattening CV accuracy as `max_depth` increased.
- Learned that for a Random Forest, the two randomness sources (bootstrapping rows vs. subsampling features per split) do not always contribute equally, on this dataset the feature subsampling was doing most of the work.
- Got a clearer sense of why accuracy alone can be misleading on an imbalanced dataset, recall on the minority (malignant) class told a much sharper story than the overall accuracy numbers did.
- Practiced picking hyperparameters purely from cross-validation performance and only touching the test set once, at the very end, for a fair final comparison.

15 References

1. Scikit-learn: Decision Trees, <https://scikit-learn.org/stable/modules/tree.html>.
2. Scikit-learn: Random Forest, <https://scikit-learn.org/stable/modules/ensemble.html#random-forests>.
3. UCI Machine Learning Repository, Wisconsin Diagnostic Breast Cancer Dataset, <https://archive.ics.uci.edu>.
4. Stephen Marsland, “Machine Learning - An Algorithmic Perspective”, 2nd Edition, Chapman and Hall/CRC Machine Learning and Pattern Recognition Series, 2015.
5. Ethem Alpaydin, “Introduction to Machine Learning”, 3rd Edition, The MIT Press, 2014.